

Module 1

Introduction to IoT

Introduction to IoT

The Internet of Things (IoT) refers to a rapidly growing network of physical objects—devices, machines, appliances, vehicles, even living beings—that are equipped with sensors, software, and connectivity. These "smart" devices can collect and share data over the internet, allowing them to interact with each other and with the environment, often with little or no human intervention.

Think of your smartwatch tracking your heartbeat, your fridge reminding you to buy milk, or traffic signals adjusting dynamically to traffic flow. That's IoT in action—invisible but intelligent, working silently to make everyday life smoother and smarter.

"The Internet of Things has the potential to change the world, just as the internet did. Maybe even more so."

— *Kevin Ashton*, the technologist who coined the term "Internet of Things"

"Everything that can be automated will be automated." — *Robert Cannon*

IoT and IoE

- ❖ IoT is a subset of IoE.
- ❖ If IoT is about "smart devices talking to each other," then IoE is about "everything (people, data, devices, processes) working together intelligently."

Four Key Components of IOT

- ❖ Device or sensor
- ❖ Connectivity
- ❖ Data processing
- ❖ Interface

Main Components Used in IoT

- ❖ Low-power embedded systems: Less battery consumption, high performance are the inverse factors that play a significant role during the design of electronic systems.
- ❖ Sensors: Sensors are the major part of any IoT application. It is a physical device that measures and detects certain physical quantities and converts it into signal

which can be provided as an input to processing or control unit for analysis purpose.

Different types of Sensors

Temperature Sensors, Image Sensors, Gyro Sensors, Obstacle Sensors, RF Sensor, IR Sensor, MQ-02/05 Gas Sensor, LDR Sensor, Ultrasonic Distance Sensor

Some examples of IoT devices include:

- Smart home devices such as thermostats, lighting systems, and security systems.
- Wearables such as fitness trackers and smartwatches.
- Healthcare devices such as patient monitoring systems and wearable medical devices.
- Industrial systems such as predictive maintenance systems and supply chain management systems.
- Transportation systems such as connected cars and autonomous vehicles

Characteristics of IoT

1. Connectivity

Connectivity is an important requirement of the IoT infrastructure. Things of IoT should be connected to the IoT infrastructure. Anyone, anywhere, anytime can connect, this should be guaranteed at all times. For example, the connection between people through Internet devices like mobile phones, and other gadgets, also a connection between Internet devices such as routers, gateways, sensors, etc.

2. Intelligence and Identity

The extraction of knowledge from the generated data is very important. For example, a sensor generates data, but that data will only be useful if it is interpreted properly. Each IoT device has a unique identity. This identification is helpful in tracking the equipment and at times for querying its status.

3. Scalability

The number of elements connected to the IoT zone is increasing day by day. Hence, an IoT setup should be capable of handling the massive expansion. The data generated as an outcome is enormous, and it should be handled appropriately.

4. Dynamic and Self-Adapting (Complexity)

IoT devices should dynamically adapt themselves to changing contexts and scenarios. Assume a camera meant for surveillance. It should be adaptable to work in different conditions and different light situations (morning, afternoon, and night).

5. Architecture

IoT Architecture cannot be homogeneous in nature. It should be hybrid, supporting different manufacturers' products to function in the IoT network. IoT is not owned by any one engineering branch. IoT is a reality when multiple domains come together.

6. Safety

There is a danger of the sensitive personal details of the users getting compromised when all his/her devices are connected to the internet. This can cause a loss to the user. Hence, data security is the major challenge. Besides, the equipment involved is huge. IoT networks may also be at risk. Therefore, equipment safety is also critical.

7. Self Configuring

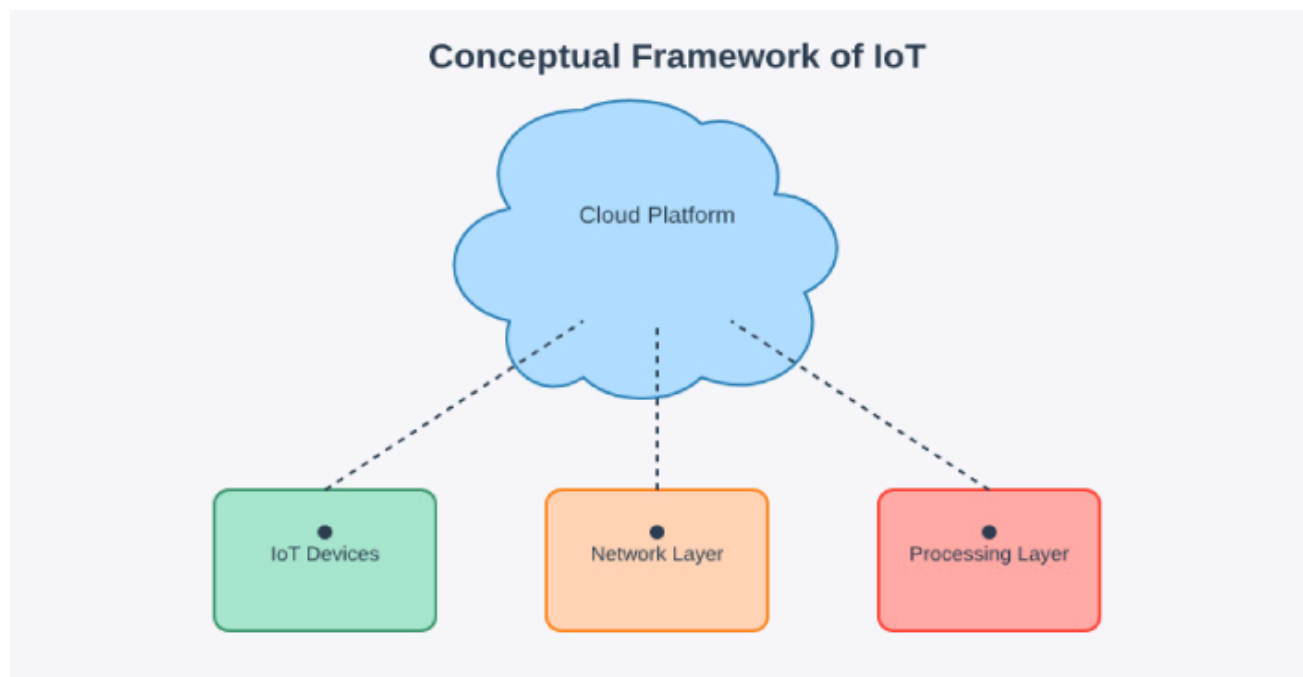
This is one of the most important characteristics of IoT. IoT devices are able to upgrade their software in accordance with requirements with a minimum of user participation. Additionally, they can set up the network, allowing for the addition of new devices to an already-existing network.

8. Interoperability

IoT devices use standardized protocols and technologies to ensure they can communicate with each other and other systems. Interoperability is one of the key characteristics of the Internet of Things (IoT). It refers to the ability of different IoT devices and systems to communicate and exchange data with each other, regardless of the underlying technology or manufacturer.

Interoperability is critical for the success of IoT, as it enables different devices and systems to work together seamlessly and provides a seamless user experience. Without interoperability, IoT systems would be limited to individual silos of data and devices, making it difficult to share information and create new services and applications.

Conceptual Framework of IoT



Connectivity Layer:

This layer comprises all the physical devices included in the framework and the established interconnectivity between them. It also includes allocating low-range networking tools like actuators in IoT, RFID tags, sensors in IoT, etc. There is also resource management which verifies the availability of devices' physical resources and networks present in the core architecture. Anyhow, these devices entail a limited number of resources and resource management makes sure that there is maximum utilization with minimum overhead. It also enables sharing and distribution of data among numerous networks. It can also share data over a network that is divided into numerous domains.

Access Layer:

Context data reaches the internet through the IoT gateway as collected by short range devices in the form of raw information. Access layer basically includes network initiation, topology definition, and creation of domains, etc. This layer also comprises intra-inter domain communication, connection setup, scheduling, and packet transmissions between IoT gateway and flow sensors. There is feature management that has a feature filter that takes in only acceptable data and automatically removes redundant data. Modern sensors have a lot of features but only a few of them can generate context data. Feature filter helps in reducing transmission of irrelevant information and increases usefulness of data. Thus, it helps in minimizing energy and CPU consumption.

Abstraction Layer:

It is a critical layer in the conceptual framework of IoT. It masks the complexities of the process and provides a simplified interface for the interaction between different components. It is an important architectural concept that converts complex technological interaction into a more simplified version by hiding the details of the sub-systems. It enables IoT developers to ensure complex interactions without having to go through the intricacies of complex technological communication.

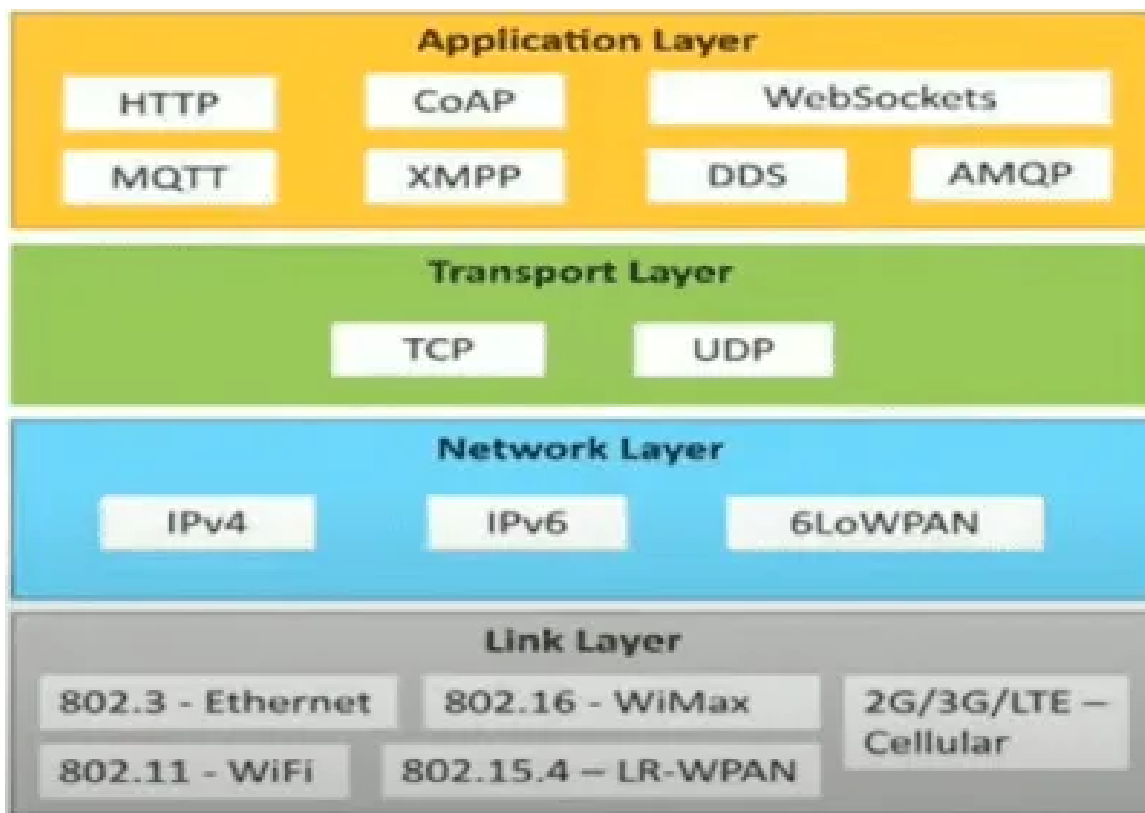
The core objectives of the abstraction layer is to minimize complexity, ensure faster integration, improve flexibility, and reduce direct dependencies on technologies. The key characteristics include management of device-level interactions, translation of protocol languages, providing support to diverse communication standards, and ensuring a uniform communication interface.

Service Layer:

This is also a vital layer responsible for storing and securing data. It also enables data accessibility effectively. Its notable characteristics include integrating information to improve service intelligence, supporting data transformation, providing service discovery, handling data processing, etc. Apart from that, it also facilitates

communication between devices. The core components of the service layer includes resource allocation, data processing services, security management, communication protocols, and service discovery mechanisms. The service layer can be implemented using cloud-enabled service platforms, edge computing services, API management, middleware solutions, etc.

Physical design of IoT



- **Application Layer Protocols**

The **Application Layer** defines how data is sent and received over the network by user applications. It interacts with the lower layers using application-specific protocols. These protocols help devices communicate with each other by following standard formats and rules. Common application layer protocols in IoT include HTTP, WebSocket, MQTT, XMPP, DDS, and AMQP, each serving specific communication needs based on the nature of the IoT application.

- **HTTP (HyperText Transfer Protocol)**

HTTP is a widely used application-layer protocol designed for transmitting media-rich documents over the web. It allows communication between web browsers (clients) and web servers using a request-response mechanism. When a client sends a request, it waits for a response from the server. Importantly, HTTP is **stateless**, meaning it does

not retain any memory of previous interactions once a transaction is complete. This protocol is ideal for web-based IoT dashboards and RESTful APIs.

- **WebSocket**

WebSocket is a communication protocol that allows **full-duplex** (two-way) communication between a client and a server. Unlike HTTP, it maintains a persistent connection, enabling data to be sent and received continuously without needing to re-establish the connection. WebSockets are commonly used in real-time applications like online gaming, live chat systems, and financial trading platforms. They are particularly suitable for web browsers running untrusted code in controlled environments.

- **MQTT (Message Queuing Telemetry Transport)**

MQTT is a lightweight, publish/subscribe-based messaging protocol specifically designed for **machine-to-machine (M2M)** communication. It is highly efficient and requires minimal bandwidth, making it ideal for remote locations or systems with limited resources. In MQTT, devices publish messages to specific topics, and other devices subscribed to those topics receive the messages. It is widely used in sensor networks, telemetry, and remote monitoring systems.

- **Transport Layer**

The **Transport Layer** is responsible for **end-to-end communication** between devices. It controls the flow of data segments, manages error detection and correction, and ensures that messages are delivered completely and correctly. This layer is independent of the underlying network and includes key protocols like TCP and UDP, which are chosen based on whether reliability or speed is more important.

- **TCP (Transmission Control Protocol)**

TCP is a **connection-oriented** protocol that guarantees reliable data transmission. It establishes a connection before sending data and ensures that all packets arrive in the correct order and without errors. If data is lost or arrives out of order, TCP handles retransmission and sequencing. This makes it ideal for applications where data integrity is crucial, such as email, file transfers, and banking systems.

- **UDP (User Datagram Protocol)**

UDP is a **connectionless** transport protocol that sends data without establishing a prior connection. It does not guarantee delivery, order, or error checking, which makes it much faster but less reliable than TCP. UDP is preferred in applications where speed is more critical than perfect accuracy, such as live video streaming, online gaming, or transmitting real-time sensor data in IoT.

- **Network Layer**

The **Network Layer** is responsible for the **routing and addressing** of data packets from the source device to the destination device across networks. It breaks down data into packets and uses IP addresses to identify and deliver data to the correct location. The most common network layer protocols are IPv4 and IPv6, which act as unique identifiers for each connected device.

- **IPv4 (Internet Protocol Version 4)**

IPv4 is a 32-bit addressing scheme that provides a unique numeric address (e.g., 192.168.0.1) to every device on a network. It performs two main functions: identifying the host (device) and providing the location (routing path). Although IPv4 has been widely used, it is limited to approximately 4.3 billion unique addresses, which is not sufficient for the explosion of IoT devices.

- **IPv6 (Internet Protocol Version 6)**

IPv6 is the next-generation IP addressing protocol that uses **128-bit addresses**, allowing for an exponentially larger number of unique IP addresses. It was developed by the IETF to overcome IPv4's limitations and to accommodate the massive scale of modern and future internet-connected devices. IPv6 also includes improved routing and security features, making it more efficient for IoT networks.

- **Link Layer**

The **Link Layer** manages communication between devices at the physical layer of the network. It is responsible for **transmitting data frames** over the actual network hardware like cables or wireless signals. It determines how data packets are coded, addressed, and signaled for transmission. Link-layer protocols ensure that data reaches the correct hardware interface on a local network.

- **Ethernet**

Ethernet is a widely used technology for **wired local area networks (LANs)**. It defines the rules for data transmission over cables and includes protocols for controlling how devices on the same network segment communicate. Ethernet is known for its high speed, stability, and reliability, making it ideal for industrial and home IoT systems that require consistent, low-latency connections.

- **Wi-Fi**

Wi-Fi is a wireless communication protocol used to create **local area networks without cables**. It uses radio signals to connect devices such as smartphones, laptops, and smart appliances to the internet. Wi-Fi operates at the link and physical layers, handling both media access control and transmission. It is essential in IoT environments like smart homes and mobile applications due to its convenience and wide availability.

Logical design of IoT

A logical design for an IoT system is the actual design of how its components (computers, sensors, and actuators) should be arranged to complete a particular function. It doesn't go into the depth of describing how each component will be built with low-level programming specifics.

IoT logical design includes:

1. IoT functional blocks
2. IoT communications models
3. IoT communication APIs

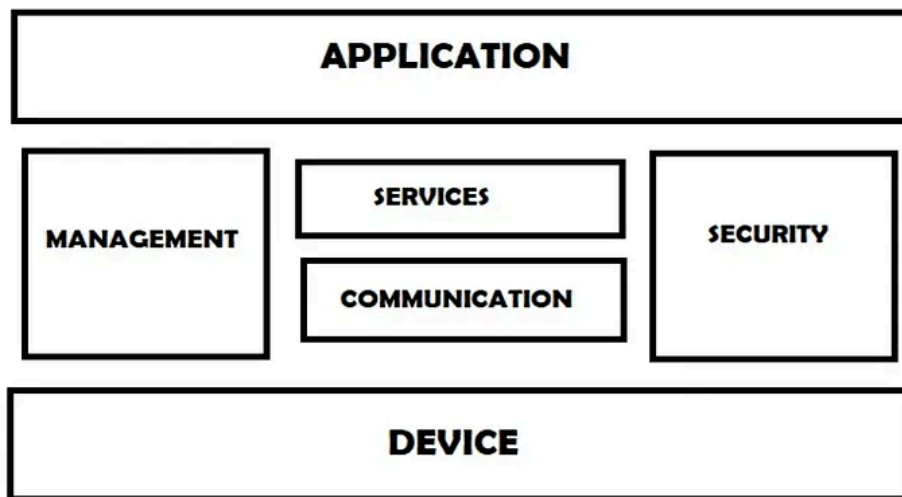
1. IoT functional blocks

IoT systems include several functional blocks such as Devices, communication, security, services, and application.

The functional blocks provide sensing, identification, actuation, management, and communication capability. These functional blocks consist of devices that handle the communication between the server and the host, enable monitoring control functions, manage the data transfer, secure the IoT system using authentication and different functions, and provide an interface for controlling and monitoring various terms.

The Functional blocks are:

- Device: An IoT system comprises of devices that provide sensing, actuation, monitoring, and control functions.
- Communication: Handles the communication for the IoT system.
- Services: services for device monitoring, device control service, data publishing services, and services for device discovery.
- Management: this block provides various functions to govern the IoT system.
- Security: This block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- Application: This is an interface that the users can use to control and monitor various aspects of the IoT system. The application also allows users to view the system status and view or analyze the processed data.



2. IoT Communication Models

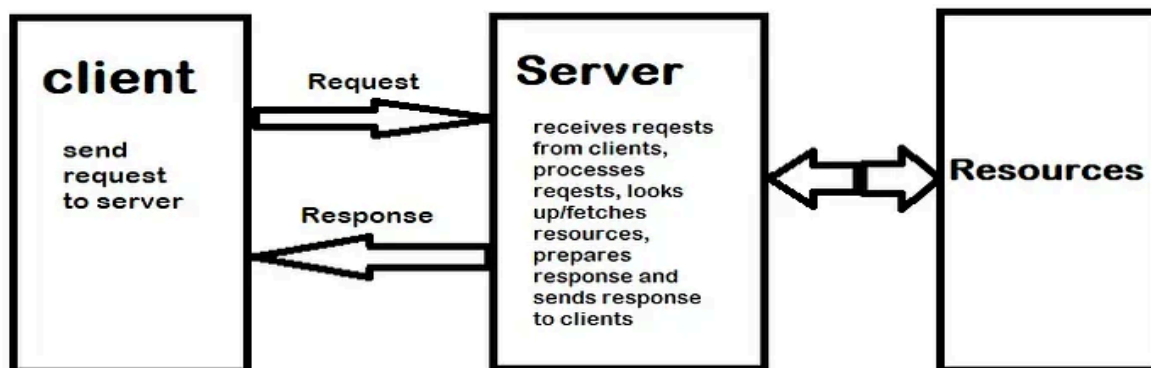
There are multiple kinds of models available in an Internet of Things system that is used for communicating between the system and server, such as:

- Request-Response Model

Request-response model is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representation, prepares the response, and then sends the response to the client. Request-response is a stateless communication model and each request-response pair is independent of the others.

HTTP works as a request-response protocol between a client and a server. A web browser may be the client, and an application on a computer that hosts a website may be the server.

Example: A client (browser) submits an HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content.



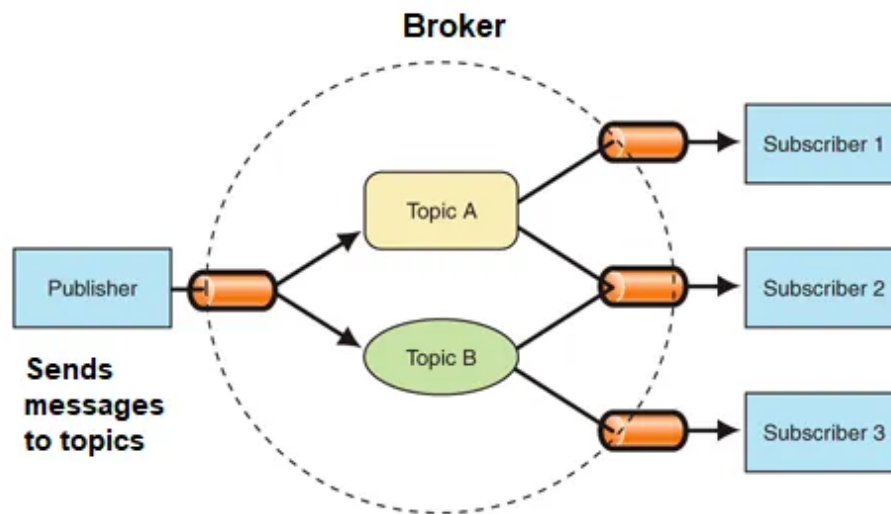
Request-Response Communication Model

- **Publisher-Subscriber Model** — This model comprises three entities: Publishers, Brokers, and Consumers.

Publishers are the source of data. It sends the data to the topic which is managed by the broker. They are not aware of consumers.

Consumers subscribe to the topics which are managed by the broker.

Brokers' responsibility is to accept data from publishers and send it to the appropriate consumers. The broker only has the information regarding the consumer to which a particular topic belongs which the publisher is unaware.



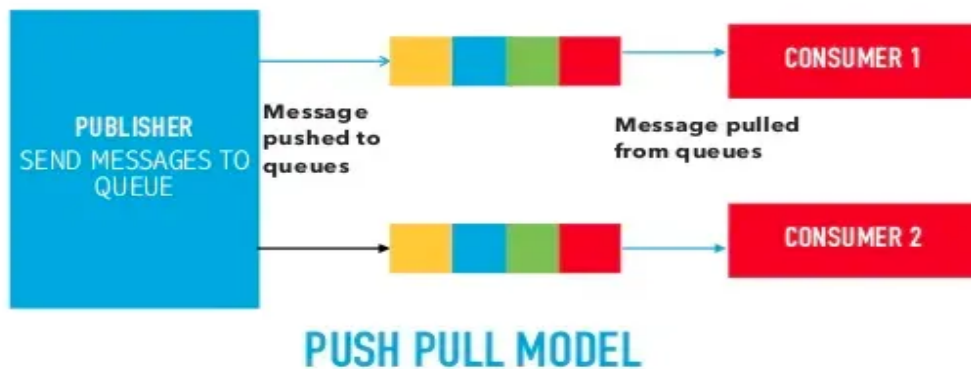
- **Push-Pull Model** — The push-pull model constitutes data publishers, data consumers, and data queues.

Publishers and Consumers are not aware of each other.

Publishers publish the message/data and push it into the queue. The consumers, present on the other side, pull the data out of the queue. Thus, the queue acts as the buffer for the message when the difference occurs in the rate of push or pull of data on the side of a publisher and consumer.

Queues help in decoupling the messaging between the producer and consumer.

Queues also act as a buffer which helps in situations where there is a mismatch between the rate at which the producers push the data and consumers pull the data.



- Exclusive Pair –

Exclusive Pair is the bi-directional model, including full-duplex communication between client and server. The connection is constant and remains open till the client sends a request to close the connection.

The Server has the record of all the connections which has been opened.

This is a state-full connection model and the server is aware of all open connections.

WebSocket-based communication API is fully based on this model.



3. IoT communication API

In IoT, there are 2 communication APIs –

- REST — based Communication APIs
- Web Socket — based Communication APIs

Web service can either be implemented using REST principles or using Web Socket Protocol –

1. REST-Based Communication API:

Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response

communication model. The REST architectural constraints apply to the components, connectors, and data elements, within a distributed hypermedia system.

2. Web Socket-Based Communication APIs:

Web Socket APIs allow bi-directional, full-duplex communication between clients and servers. It follows the exclusive pair communication model. This Communication API does not require a new connection to be set up for each message to be sent between clients and servers. Once the connection is set up the messages can be sent and received continuously without any interruption. WebSocket APIs are suitable for IoT Applications with low latency or high throughput requirements.

Functional blocks of IoT

- **Device**

In an IoT system, the **device layer** forms the foundation and consists of the physical components such as sensors and actuators. These devices are responsible for sensing environmental data (like temperature, humidity, motion) and performing actions (such as switching lights on/off or adjusting machinery). They play a crucial role in **monitoring, controlling, and interacting with the physical world**, forming the eyes, ears, and hands of an IoT system.

- **Communication**

The **communication block** ensures reliable data exchange between IoT devices and other components in the system, such as gateways, servers, or cloud platforms. It supports both **wired and wireless communication protocols** (e.g., Wi-Fi, ZigBee, Bluetooth, LTE) to transmit sensor data and receive commands. This layer acts as the central nervous system, enabling seamless data flow throughout the network.

- **Services**

The **services layer** provides essential functionalities that support device interaction and data handling. These services include **device monitoring**, which tracks device health and performance; **device control**, which allows users or systems to send instructions; **data publishing**, which shares collected data with subscribers or cloud applications; and **device discovery**, which helps identify and connect new or existing devices in the network.

- **Management**

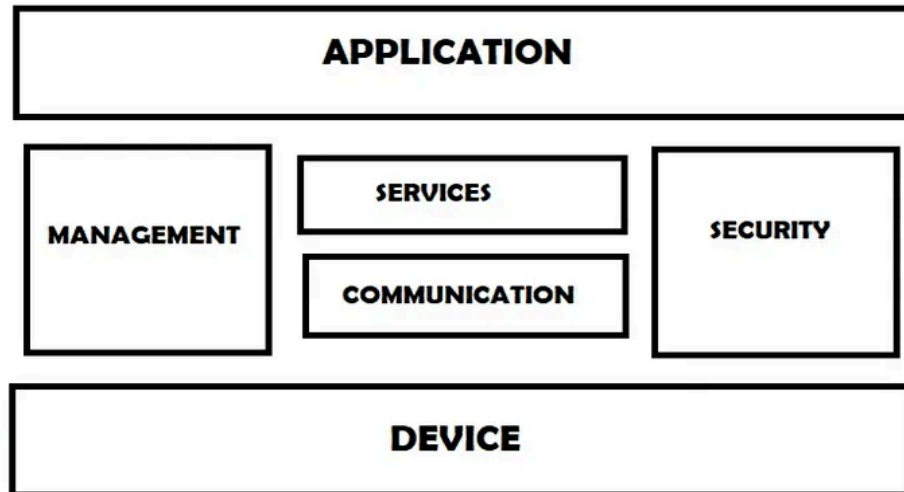
The **management block** oversees the operation and configuration of the IoT system. It includes functions such as **device provisioning**, firmware updates, performance monitoring, and fault detection. Effective management ensures that the IoT system remains efficient, scalable, and responsive to changing requirements or failures. This block is crucial for maintaining the long-term reliability of the system.

- **Security**

The **security layer** is responsible for protecting the IoT ecosystem from threats and unauthorized access. It provides key features such as **authentication** (verifying device or user identity), **authorization** (controlling access levels), **message and content integrity** (ensuring data is not tampered with), and **data encryption** (safeguarding information during transmission and storage). Security is fundamental to building trust and resilience in IoT deployments.

- **Application**

The **application layer** serves as the **user interface** to the IoT system. It allows users to **monitor the system's status, control devices, and visualize or analyze data** collected from various sources. This layer could be a mobile app, web dashboard, or any graphical interface that provides insights and interactive controls. Ultimately, the application connects human users to the intelligence of the IoT system.



Brief review of applications of IoT

1. Smart Homes

IoT enables automation and remote control of home appliances like lighting, air conditioning, security systems, and smart TVs. Devices can be controlled via smartphones or voice assistants, enhancing convenience, energy efficiency, and security.

2. Healthcare

IoT is widely used in remote patient monitoring, wearable health devices, and smart medical equipment. It allows real-time tracking of vital signs, medication reminders, and emergency alerts, improving patient care and hospital management.

3. Industrial IoT (IIoT)

In manufacturing and production industries, IoT improves operational efficiency through predictive maintenance, real-time monitoring of machinery, and automation. It reduces downtime and enhances productivity through data-driven decision-making.

4. Smart Cities

IoT contributes to intelligent urban infrastructure by enabling smart traffic management, waste management, energy usage optimization, pollution monitoring, and public safety systems. This leads to better resource management and a higher quality of life.

5. Agriculture

IoT in agriculture involves precision farming, automated irrigation, soil health monitoring, and livestock tracking. These technologies help increase crop yield, reduce water usage, and optimize farm operations.

6. Transportation and Logistics

IoT supports vehicle tracking, fleet management, real-time traffic updates, and predictive maintenance in transportation. In logistics, it enhances supply chain visibility, cold-chain monitoring, and package tracking.

7. Retail

Retailers use IoT for inventory management, smart shelves, personalized marketing, and customer behavior analysis. It enhances the shopping experience and streamlines store operations.

Smart Object

- **Definition of Smart Object**

A Smart Object in the context of IoT is a physical object embedded with sensors, actuators, software, and communication capabilities that allow it to sense its environment, process data, and communicate with other devices or systems over a network.

In simple terms, a smart object is an enhanced version of a regular object that can collect data, make decisions, and take action autonomously or based on external commands.

- **Characteristics of smart object**

Sensing Capabilities: Smart objects are equipped with sensors that allow them to gather data about their internal state or their surrounding environment. This can include temperature, pressure, light, motion, humidity, location, and more.

Processing Unit (Microprocessor/Microcontroller): They contain a small processing unit that enables them to process the data collected by their sensors, execute pre-programmed algorithms, and make decisions locally. This allows for decentralized intelligence and real-time responsiveness.

Communication Interfaces: Smart objects possess communication capabilities, typically wireless (e.g., Wi-Fi, Bluetooth, RFID), that enable them to exchange data with other smart objects, central platforms, or users. This connectivity is fundamental to their integration into IoT networks.

Actuators (Optional but Common): Many smart objects also include actuators, which allow them to perform physical actions or control other devices based on the processed data or received commands. Examples include turning lights on/off, adjusting thermostats, or unlocking doors.

Power Source: They require a power source, which can be a battery, a direct power supply, or even energy harvesting mechanisms (for ambient IoT). The power consumption is often optimized for prolonged operation.

Connectivity/Networking: Smart objects are designed to be integrated into networks, often forming an IoT ecosystem. They can communicate directly with each other (device-to-device) or with a central hub/cloud platform.

Autonomy and Self-Governance: While not always fully autonomous, smart objects can often operate independently to some extent, collecting data, processing it, and initiating actions without constant human intervention.

Data Collection, Storage, and Processing: They can collect, store (often temporarily), and process data. This data can then be analyzed to gain insights, optimize operations, and improve performance.

Interaction: Smart objects are designed to interact not only with other smart objects and systems but also with human users, providing input, output, control, and feedback.

Reconfigurability: Many smart objects can be reconfigured or updated remotely, allowing for new functionalities or adjustments to their behavior over time.

- **Trends**

- Green IoT (GloT)
- Massive IoT
- Digital Twins
- Enhanced Connectivity: 5G and Beyond (including LPWAN and Wi-Fi 7)